

ЗВІТ

з лабораторної роботи № 1

з дисципліни «Проектування, розробка і реалізація криптографічних систем»
на тему: «Дослідження реалізацій протоколу SSL/TLS»

Виконала:
студентка групи ФІ-52мн
Балацька Вікторія

Київ — 2026

1. Мета роботи

Дослідження особливостей реалізації криптографічних механізмів протоколу TLS, принципів встановлення захищеного з'єднання, використання цифрових сертифікатів та механізмів протидії атакам на захищений канал зв'язку.

2. Використані програмні засоби

Засіб	Версія / режим	Призначення в роботі
OpenSSL (openssl s_client, openssl x509)	3.0.13 (30 Jan 2024), Linux x86_64	Встановлення TLS-з'єднання, отримання ланцюжка сертифікатів, узгодженої версії протоколу, набору шифрів, параметрів обміну ключами; розбір полів X.509.
Qualys SSL Labs API	v3 (/api/v3/analyze)	Незалежне зовнішнє сканування сервера: повний ланцюжок сертифікатів, підтримувані версії протоколу та набори шифрів, політика HSTS, наявність SCT.
Сховище корневих сертифікатів Mozilla (пакет Python certifi)	актуальна версія пакета	Еталонний набір публічно довірених корневих ЦС для перевірки ланцюжка, незалежний від системного сховища.
Веббраузер (панель «Безпека» DevTools / перегляд сертифіката)	Chromium-сумісний	Візуальне підтвердження версії TLS, шифру та ланцюжка сертифіката (рекомендується для знімків екрана).

Для дослідження обрано два загальнодоступні HTTPS-ресурси: **github.com** та **pypi.org**. Використані команди:

```
# Встановлення з'єднання, вивід ланцюжка сертифікатів і параметрів сесії
openssl s_client -connect github.com:443 -servername github.com -showcerts </dev/null

# Короткий підсумок рукописання (версія, шифр, тип підпису, ключ обміну)
openssl s_client -connect github.com:443 -servername github.com -brief </dev/null

# Примусове використання TLS 1.2 та перевірка відмови від TLS 1.1
openssl s_client -connect github.com:443 -servername github.com -tls1_2 </dev/null
openssl s_client -connect github.com:443 -servername github.com -tls1_1 </dev/null

# Перевірка ланцюжка лише щодо корневих ЦС зі сховища Mozilla
openssl s_client -connect github.com:443 -servername github.com \
    -CAfile $(python3 -c "import certifi;print(certifi.where())") </dev/null

# Розбір полів сертифіката сервера (SAN, ключ, підпис, строк дії)
openssl s_client -connect pypi.org:443 -servername pypi.org </dev/null 2>/dev/null \
    | openssl x509 -noout -subject -issuer -dates -ext subjectAltName -text
```

3. Результати практичного дослідження TLS-з'єднань

3.1. Ресурс github.com

Параметри з'єднання, отримані під час рукописання з клієнтом OpenSSL:

```
$ openssl s_client -connect github.com:443 -servername github.com -brief </dev/null
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
```

```
Peer certificate: CN = github.com
Hash used: SHA256
Signature type: RSA-PSS
Verification: OK
Server Temp Key: X25519, 253 bits
```

Лістинг 1 — Узгоджені параметри TLS-сесії з github.com

```
$ openssl s_client -connect github.com:443 -servername github.com -tls1_2 </dev/null
Server Temp Key: X25519, 253 bits
New, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
    Protocol  : TLSv1.2

$ openssl s_client -connect github.com:443 -servername github.com -tls1_1 </dev/null
error:0A0000BF:SSL routines:tls_setup_handshake:no protocols available
New, (NONE), Cipher is (NONE)
```

Лістинг 2 — Перевірка підтримки TLS 1.2 та відмова від TLS 1.1

Зовнішнє сканування SSL Labs (фрагмент відповіді API, скорочено):

```
"protocols": [{"name": "TLS", "version": "1.2"}, {"name": "TLS", "version": "1.3"}]
TLS 1.3 suites: TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256,
                TLS_AES_256_GCM_SHA384 (група обміну ключами: x25519)
TLS 1.2 suites: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256, ..._CHACHA20_POLY1305_SHA256,
                TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384, ...
certs[0]: subject "CN=github.com"
          altNames ["github.com", "www.github.com"]
          notBefore 1785801600000 notAfter 1793577599000
          issuerSubject "CN=Sectigo Public Server Authentication CA DV R36, O=Sectigo Limited, C=GB"
          sigAlg "SHA256withRSA" keyAlg "RSA" keySize 4096
          ocspURIs ["http://ocsp.sectigo.com"] sct true
certs[1]: subject "CN=Sectigo Public Server Authentication CA DV R36, O=Sectigo Limited, C=GB"
          issuerSubject "CN=Sectigo Public Server Authentication Root R46, O=Sectigo Limited, C=GB"
          sigAlg "SHA384withRSA" keyAlg "RSA" keySize 3072
          crlURIs ["http://crl.sectigo.com/SectigoPublicServerAuthenticationRootR46.crl"]
hstsPolicy.header: "max-age=31536000; includeSubdomains; preload"
```

Лістинг 3 — Результат сканування github.com через SSL Labs API v3

Параметр	Значення для github.com
Версія протоколу	TLS 1.3 (сервер також підтримує TLS 1.2; TLS 1.1 і нижче відхиляються)
Cipher suite (TLS 1.3)	Перевага сервера — TLS_AES_128_GCM_SHA256; також TLS_CHACHA20_POLY1305_SHA256, TLS_AES_256_GCM_SHA384
Обмін ключами	ECDHE на кривій X25519 (ефемерний ключ, 253 біти)
Суб'єкт (Subject)	CN=github.com
Видавець (Issuer)	Sectigo Public Server Authentication CA DV R36, O=Sectigo Limited, C=GB (сертифікат DV — перевірено лише контроль над доменом)
Строк дії	04.08.2026 00:00:00 UTC — 01.11.2026 23:59:59 UTC (≈ 90 діб)
SAN	DNS:github.com, DNS:www.github.com
Відкритий ключ	RSA 4096 біт (GitHub паралельно обслуговує ECDSA-сертифікат для клієнтів, що його підтримують — друга гілка ланцюжка у звіті SSL Labs і набори ECDHE_ECDSA у TLS 1.2)
Алгоритм підпису сертифіката	sha256WithRSAEncryption (підпис проміжного ЦС)
Підпис у рукописанні (CertificateVerify)	RSA-PSS з SHA-256

Параметр	Значення для github.com
Статус / CT	OCSF: http://ocsp.sectigo.com; сертифікат містить SCT (Certificate Transparency)
HSTS	max-age=31536000; includeSubdomains; preload

Таблиця 1 — Параметри TLS-з'єднання та сертифіката github.com

Ланцюжок сертифікації github.com:

Рівень	Сертифікат	Ключ / підпис	Ким підписаний
0 — кінцевий (leaf)	CN=github.com	RSA 4096 / SHA256withRSA	Sectigo ... CA DV R36
1 — проміжний ЦС	Sectigo Public Server Authentication CA DV R36 (дійсний 22.03.2021 — 21.03.2036)	RSA 3072 / SHA384withRSA	Sectigo ... Root R46
2 — кореневий ЦС (довірений)	Sectigo Public Server Authentication Root R46	самопідписаний	міститься в сховищах довірених коренів ОС і браузерів; сервером не передається

Таблиця 2 — Ланцюжок сертифікації github.com

3.2. Ресурс pypi.org та виявлення TLS-інспекції

Під час з'єднання з pypi.org (і, як з'ясувалося, з github.com) із середовища, у якому виконувалася робота, клієнт OpenSSL отримав ланцюжок, **відмінний від справжнього**, — сертифікат видано не публічним ЦС, а внутрішнім ЦС шлюзу вихідного трафіку:

```
$ openssl s_client -connect pypi.org:443 -servername pypi.org -showcerts </dev/null
Certificate chain
 0 s:CN = pypi.org
   i:O = Anthropic, CN = Egress Gateway SDS Issuing CA (production)
   a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
   v:NotBefore: Sep 21 19:05:14 2026 GMT; NotAfter: Oct 21 19:06:14 2026 GMT
 1 s:O = Anthropic, CN = Egress Gateway SDS Issuing CA (production)
   i:O = Anthropic, CN = sandbox-egress-gateway-production Egress Gateway CA
 2 s:O = Anthropic, CN = sandbox-egress-gateway-production Egress Gateway CA
   i:O = Anthropic, CN = sandbox-egress-gateway-production Egress Gateway CA
---
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
Verification: OK
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
X509v3 Subject Alternative Name:
    DNS:pypi.org
```

Лістинг 4 — Ланцюжок, отриманий під час з'єднання з pypi.org через шлюз

```
$ openssl s_client -connect github.com:443 -servername github.com \
  -CAfile ../certifi/cacert.pem -verify_return_error </dev/null
verify error:num=19:self-signed certificate in certificate chain
Verification error: self-signed certificate in certificate chain
Verify return code: 19 (self-signed certificate in certificate chain)
```

Параметр	Значення (pypi.org, з'єднання через шлюз)
Версія протоколу / шифр	TLS 1.3, TLS_AES_256_GCM_SHA384
Обмін ключами / підпис рукописання	ECDHE X25519 / RSA-PSS (SHA-256)
Суб'єкт	CN=pypi.org
Видавець	O=Anthropic, CN=Egress Gateway SDS Issuing CA (production)
Строк дії	21.09.2026 — 21.10.2026 (30 діб; сертифікат генерується шлюзом автоматично)
SAN	DNS:pypi.org (лише одне ім'я — рівно те, що запитав клієнт у SNI)
Ключ / підпис сертифіката	RSA 2048 / sha256WithRSAEncryption
Ланцюжок	pypi.org → Egress Gateway SDS Issuing CA → sandbox-egress-gateway Egress Gateway CA (самопідписаний корінь, установлений у системне сховище середовища)
Результат перевірки	ОК — щодо системного сховища; помилка 19 — щодо публічного сховища Mozilla

Таблиця 3 — Параметри з'єднання з pypi.org, отримані в середовищі з TLS-інспекцією

Аналіз. Порівняння з незалежним скануванням SSL Labs (п. 3.1) показує, що справжній сертифікат github.com — RSA 4096 від Sectigo зі строком 90 діб, тоді як локальний клієнт отримав RSA 2048 від іншого ЦС зі строком 30 діб. Отже, між клієнтом і сервером розташований проміжний вузол, який завершує TLS-сесію клієнта, генерує «на льоту» сертифікат на запитане в SNI ім'я і встановлює окреме TLS-з'єднання з реальним сервером. Такий вузол є легітимним «проксі з TLS-інспекцією»: він працює лише тому, що його кореневий сертифікат навмисно додано до сховища довірених ЦС у цьому середовищі. Для клієнта, який довіряє лише публічним кореням (лістинг 5), той самий ланцюжок відхиляється. Цей результат наочно демонструє, що **вся безпека TLS щодо автентифікації сервера тримається на переліку довірених корневих ЦС і на тому, що клієнт справді виконує перевірку**, — це безпосередньо пов'язано з ситуацією, розглянутою в розділі 6.

Параметри TLS-сесії (TLS 1.3, AEAD-шифр, X25519, RSA-PSS) у цьому випадку характеризують з'єднання клієнта зі шлюзом, а не з сервером pypi.org. Параметри справжнього ланцюжка pypi.org слід отримати з мережі без TLS-інспекції тими самими командами та порівняти з таблицею 3.

3.3. Призначення досліджених параметрів

Параметр	Призначення
Версія протоколу	Визначає набір допустимих механізмів: TLS 1.3 вилучає застарілі алгоритми та шифрує більшу частину рукописання.
Cipher suite	У TLS 1.3 задає AEAD-алгоритм (AES-GCM або ChaCha20-Poly1305) і хеш-функцію для HKDF; обмін ключами та підпис узгоджуються

Параметр	Призначення
	окремо. У TLS 1.2 набір задає також обмін ключами та автентифікацію (напр., ECDHE-RSA).
Server Temp Key (X25519)	Ефемерний ключ ECDHE, з якого виводиться спільний секрет; забезпечує Forward Secrecy.
Subject / Issuer	Subject — кому належить відкритий ключ; Issuer — ЦС, що своїм підписом засвідчив цю прив'язку. За Issuer будується ланцюжок до кореня.
Строк дії (notBefore/notAfter)	Обмежує час використання ключа й вплив його можливої компрометації. Короткі строки (90 діб у GitHub) зменшують залежність від механізмів відкликання.
SAN	Перелік імен (DNS-імен, IP-адрес), для яких дійсний сертифікат. Сучасні клієнти перевіряють ім'я хоста саме за SAN, а не за CN (RFC 6125, вимоги CA/B Forum).
Алгоритм відкритого ключа	RSA/ECDSA-ключ сервера використовується для підпису CertificateVerify, що доводить володіння закритим ключем.
Алгоритм підпису сертифіката	Алгоритм, яким ЦС підписав сертифікат; його стійкість визначає неможливість підробити сертифікат.
Ланцюжок сертифікації	Послідовність «сертифікат сервера → проміжні ЦС → довірений корінь», що дозволяє клієнту перевірити сертифікат, маючи лише кореневі сертифікати.

Таблиця 4 — Призначення параметрів TLS-з'єднання та сертифіката

3.4. Етапи встановлення TLS-з'єднання (на прикладі TLS 1.3)

Отримані дані (TLS 1.3, X25519, RSA-PSS, AEAD-шифр) відповідають повному рукостисканню TLS 1.3 за RFC 8446, яке виконується за один обмін (1-RTT):

Крок	Повідомлення	Що відбувається
1	ClientHello (відкрито)	Клієнт надсилає випадкове число, списки підтримуваних версій (supported_versions), наборів шифрів, груп і алгоритмів підпису, ім'я сервера (SNI) та одразу свою частку ECDHE (key_share, X25519).
2	ServerHello (відкрито)	Сервер обирає версію TLS 1.3, набір шифрів і групу X25519, надсилає свою частку key_share. Обидві сторони обчислюють спільний секрет ECDHE.
3	Розклад ключів	Через HKDF зі спільного секрету і хешу стенограми виводяться handshake-ключі. Усі подальші повідомлення рукостискання шифруються.
4	{EncryptedExtensions}	Додаткові параметри (ALPN тощо).
5	{Certificate}	Сервер передає ланцюжок сертифікатів (leaf + проміжні).
6	{CertificateVerify}	Підпис закритим ключем сервера (у нас — RSA-PSS) над хешем усієї стенограми рукостискання.
7	{Finished} сервера і клієнта	НМАС над стенограмою на ключах, виведених із секрету; підтверджує, що обидві сторони бачили однакові повідомлення.
8	Прикладні дані	З master secret виводяться трафікові ключі; дані захищаються AEAD (AES-GCM/ChaCha20-Poly1305).

Автентифікація сервера забезпечується поєднанням двох перевірок: (1) клієнт будує та перевіряє ланцюжок сертифікатів до довіреного кореня, перевіряє строк дії, статус і відповідність імені хоста полю SAN; (2) перевіряє підпис CertificateVerify відкритим ключем із сертифіката — це доводить, що співрозмовник володіє відповідним закритим ключем, а не просто переслав чужий сертифікат. Оскільки підписується хеш стенограми, що містить ефемерні ключі обох сторін, підпис неможливо повторно використати в іншій сесії.

Узгодження криптографічних параметрів відбувається через ClientHello/ServerHello: клієнт пропонує, сервер обирає. Захист від підміни пропозицій забезпечується повідомленнями Finished (MAC над усією стенограмою) і підписом CertificateVerify; крім того, TLS 1.3 вбудовує в ServerHello.random спеціальний маркер при відкаті до TLS 1.2, що дозволяє виявити атаку downgrade.

Формування ключового матеріалу: спільний секрет ECDHE (X25519) → HKDF-Extract/Expand (хеш SHA-256 або SHA-384 залежно від набору) → окремі секрети рукостискання й трафіку для кожного напрямку. Ефемерні ключі знищуються після сесії, тому компрометація довготривалого ключа сервера не дозволяє розшифрувати записаний трафік.

4. Відповіді на питання щодо сучасних механізмів TLS

4.1. Принципові відмінності TLS 1.3 від TLS 1.2

- **Скорочене рукостискання:** 1-RTT замість 2-RTT; можливий режим 0-RTT при відновленні сесії (з ризиком повторного відтворення ранніх даних).
- **Лише Forward Secrecy:** статичний обмін ключами RSA і статичний DH вилучено; єдиний варіант — ефемерний (EC)DHE або PSK з (EC)DHE.
- **Вилучено застарілі алгоритми:** CBC-режими, RC4, 3DES, MD5, SHA-1 у підписах, компресія, повторне узгодження (renegotiation), експортні шифри. Залишено лише AEAD: AES-GCM, AES-CCM, ChaCha20-Poly1305.
- **Спрощені набори шифрів:** 5 наборів замість сотень; набір задає лише AEAD і хеш, а обмін ключами та підпис узгоджуються окремими розширеннями.
- **Шифрування рукостискання:** після ServerHello все, включно з сертифікатом сервера, передається зашифрованим.
- **Новий розклад ключів на основі HKDF** замість PRF TLS 1.2; підпис сервера охоплює всю стенограму, у підписах RSA використовується лише RSA-PSS.
- **Вбудований захист від downgrade** через маркер у ServerHello.random.

4.2. Чому SSL та ранні версії TLS не повинні використовуватися

SSL 2.0 і SSL 3.0 заборонено відповідно RFC 6176 та RFC 7568, TLS 1.0 і 1.1 — RFC 8996 (2021). Причини: вразливі конструкції, які неможливо виправити без зміни протоколу, — недетермінований padding у SSL 3.0 (POODLE), передбачуваний IV у CBC у TLS 1.0 (BEAST), MAC-then-Encrypt (Lucky 13), залежність PRF і підписів рукописання від MD5/SHA-1, відсутність AEAD, підтримка експортних та слабких шифрів (FREAK, Logjam), слабкий захист від downgrade. Крім того, ці версії не відповідають вимогам PCI DSS і NIST SP 800-52 Rev. 2, а сучасні браузері їх вимкнули. Навіть підтримка старої версії «для сумісності» відкриває шлях атакам downgrade.

4.3. Forward Secrecy

Forward Secrecy (пряма секретність) — властивість, за якої сеансові ключі не можна відновити з довготривалого ключа сервера. Досягається ефемерним обміном ключами (ECDHE): для кожної сесії генерується нова пара ключів, яку після завершення знищують. Захищає від такого сценарію: зловмисник записує зашифрований трафік, а згодом отримує закритий ключ сервера (крадіжка, Heartbleed, вилучення, помилка адміністратора) — без FS (статичний RSA-обмін у TLS 1.2) він розшифрував би всі записані сесії, з FS — ні. FS не захищає поточні та майбутні сесії від активного MITM зі вкраденим ключем, поки сертифікат не відкликано.

4.4. Призначення Certificate Transparency

Certificate Transparency (RFC 6962, RFC 9162) — система публічних журналів лише для додавання (append-only, на основі дерев Меркла), куди ЦС зобов'язані заносити кожен виданий сертифікат. ЦС отримує від журналу підписане зобов'язання SCT і вбудовує його в сертифікат (у `github.com sct: true`); Chrome і Safari не приймають публічних сертифікатів без достатньої кількості SCT. Мета — зробити видачу сертифікатів відкритою для аудиту: власник домену може відстежувати (напр., через `crt.sh`) усі сертифікати на свої імена і виявити помилково або зловмисно виданий сертифікат (як у випадках DigiNotar, Symantec). СТ не запобігає видачі, а забезпечує її швидке виявлення.

4.5. CRL та OCSP: використання і обмеження

CRL (RFC 5280) — підписаний ЦС список серійних номерів відкликаних сертифікатів, який клієнт завантажує за адресою з розширення CRL Distribution Points (у проміжного ЦС `github.com` — `crl.sectigo.com`). **OCSP** (RFC 6960) — онлайн-запит статусу одного сертифіката до OCSP-респондера (`ocsp.sectigo.com`); відповідь «good/revoked/unknown» підписана ЦС. **OCSP stapling** дозволяє серверу самому додати свіжу відповідь OCSP у рукописання.

- CRL великі, оновлюються періодично — затримка між відкликанням і його виявленням.
- OCSP розкриває ЦС, який сайт відвідує користувач (проблема приватності), додає затримку і створює залежність від доступності респондера.
- Браузери зазвичай застосовують «soft-fail»: якщо статус отримати не вдалося, сертифікат приймається — атакувальник, що контролює мережу, може просто заблокувати запит.
- Через ці вади браузері перейшли на власні агреговані списки (CRLSets у Chrome, CRLite у Firefox), а Let's Encrypt у 2025 р. припинив підтримку OCSP. Основним засобом обмеження ризику стають короткі строки дії сертифікатів (CA/B Forum поетапно зменшує максимальний строк до 47 діб до 2029 р.).

4.6. HTTP Strict Transport Security (HSTS)

HSTS (RFC 6797) — заголовок відповіді Strict-Transport-Security, яким сервер повідомляє браузеру, що протягом max-age секунд до домену (і за includeSubDomains — до піддоменів) можна звертатися лише через HTTPS. Браузер автоматично замінює http:// на https:// і **не дозволяє користувачу ігнорувати помилку сертифіката**. Захищає від атак SSL stripping (sslstrip), при яких MITM утримує жертву на HTTP-версії сайту, від витоку cookie через незахищені запити та від «проклікування» попереджень про підроблений сертифікат. Проблема першого візиту вирішується списком HSTS preload, вбудованим у браузері (github.com має preload, max-age 1 рік).

4.7. Навіщо перевіряти сертифікат, якщо дані шифруються

Шифрування захищає дані лише від тих, хто не знає сеансового ключа. Обмін ключами ECDHE сам по собі **не автентифікований**: він безпечно встановлює спільний ключ з тим, хто відповідає, — але не гарантує, що відповідає справжній сервер. Без перевірки сертифіката зломисник між клієнтом і сервером проводить два окремі обміни ключами (з клієнтом і з сервером), розшифровує, читає, змінює й повторно шифрує трафік. Клієнт бачить «зашифроване з'єднання», але з атакувальником. Саме перевірка ланцюжка, імені хоста та підпису CertificateVerify прив'язує ключ сесії до ідентичності сервера. Результат п. 3.2 є прямою демонстрацією: з'єднання з «ruri.org» було зашифроване TLS 1.3, але кінцевою точкою був шлюз; відрізнити його від справжнього сервера дозволяє лише перевірка сертифіката.

5. Дослідження вразливості: POODLE

Для дослідження обрано атаку **POODLE** (Padding Oracle On Downgraded Legacy Encryption, CVE-2014-3566), опубліковану дослідниками Google (Möller, Duong, Kotowicz) у жовтні 2014 р.

5.1. Сутність атаки

POODLE — атака типу padding oracle на режим CBC у протоколі SSL 3.0, поєднана з примусовим відкатом (downgrade) з'єднання до SSL 3.0. Дозволяє розшифровувати секретні фрагменти трафіку (найчастіше cookie сесії) по одному байту, у середньому за 256 запитів на байт.

5.2. Умови реалізації

- Клієнт і сервер підтримують SSL 3.0 з блоковим шифром у режимі CBC (у 2014 р. — більшість систем).
- Атакувальник займає позицію MITM (напр., спільна Wi-Fi-мережа) і може змусити клієнта відкотитися до SSL 3.0, обриваючи спроби рукостискання TLS 1.x (браузери того часу автоматично повторювали з'єднання з нижчою версією — «fallback dance»).
- Атакувальник може змусити браузер жертви надсилати багато запитів до цільового сайту з керованою довжиною шляху/тіла (напр., шкідливий JavaScript на іншій сторінці), щоб секретний байт опинявся в потрібній позиції блоку.

5.3. Особливість протоколу, що зробила атаку можливою

У SSL 3.0 використовується схема MAC-then-Encrypt: MAC обчислюється над відкритим текстом, потім додається padding і все шифрується в CBC. Padding у SSL 3.0 **не детермінований і не захищений MAC**: перевіряється лише останній байт (довжина доповнення), решта байтів доповнення довільні. Якщо запит підібрано так, що останній блок повністю складається з доповнення, атакувальник замінює його на блок шифротексту, що містить секретний байт. Сервер приймає запис лише тоді, коли після розшифрування останній байт дорівнює очікуваній довжині (ймовірність 1/256); прийняття/відхилення запису є «оракулом», з якого за формулою CBC відновлюється один байт відкритого тексту. Друга передумова — **відсутність у протоколі надійного захисту від downgrade**, що дозволяло вимусити використання SSL 3.0 у клієнтів і серверів, які підтримували новіші версії.

5.4. Можливі наслідки

Розкриття cookie сесії, токенів автентифікації, заголовків Authorization — і як наслідок, перехоплення облікового запису (session hijacking) без знання пароля. Цілісність і конфіденційність усієї сесії фактично втрачаються. Варіант POODLE-TLS (CVE-2014-8730) вразив також реалізації TLS 1.0–1.2 (деякі балансувальники F5, A10), які не перевіряли вміст padding.

5.5. Усунення та протидія

- Повне вимкнення SSL 3.0 на серверах і в браузерах (Chrome 40, Firefox 34/35, IE 11 — кінець 2014 — початок 2015 р.); формальна заборона — RFC 7568 (2015).

- Механізм TLS_FALLBACK_SCSV (RFC 7507): клієнт при повторній спробі з нижчою версією додає сигнальний шифронабір, і сервер, що підтримує вищу версію, обриває з'єднання — атаку downgrade виявлено.
- Відмова від CBC-наборів на користь AEAD (AES-GCM, ChaCha20-Poly1305), виправлення перевірки padding у реалізаціях TLS (проти POODLE-TLS).
- Перевірка конфігурації: `openssl s_client -ssl3` повинен завершуватися помилкою; сучасні збірки OpenSSL взагалі не містять SSL 3.0. Для `github.com` підтримка навіть TLS 1.1 відсутня (лістинг 2).

5.6. Актуальність для TLS 1.3

Для TLS 1.3 атака **неактуальна**: протокол не підтримує SSL 3.0 і режим CBC, використовує лише AEAD-шифри, де цілісність шифротексту перевіряється до обробки відкритого тексту (padding oracle неможливий), а захист від downgrade вбудовано в `ServerHello.random` і підпис стенограми. Ризик зберігається лише для застарілих систем, у яких досі ввімкнено SSL 3.0 або вразливі реалізації CBC у TLS 1.0–1.2.

6. Аналіз ситуації порушення безпеки

Ситуація: розробник клієнтського застосунку отримав помилку перевірки сертифіката сервера і вимкнув перевірку (напр., `verify=False` у Python `requests`, `InsecureSkipVerify: true` у Go, `curl -k`, порожній `X509TrustManager` у Java). З'єднання встановлюється, дані передаються зашифрованими.

6.1. Чому після вимкнення перевірки з'єднання стало можливим

Помилка виникала на етапі автентифікації сервера: клієнт не зміг побудувати довірений ланцюжок або перевірити відповідність імені. Типові причини — самопідписаний або внутрішній сертифікат, сервер не передає проміжний сертифікат, прострочений сертифікат, невідповідність імені хоста SAN, застаріле сховище кореневих ЦС клієнта, корпоративний проксі з TLS-інспекцією (як у п. 3.2). Вимкнення перевірки не усунуло причину — воно лише змусило клієнт ігнорувати результат перевірки. Обмін ключами ECDHE та шифрування працюють незалежно від неї, тому рукописання завершується успішно з будь-яким співрозмовником.

6.2. Які властивості каналу зберігаються, а які порушуються

Властивість	Стан	Пояснення
Конфіденційність щодо пасивного спостерігача	Зберігається	Той, хто лише прослуховує трафік, не може його розшифрувати.
Цілісність щодо пасивного спостерігача / випадкових помилок	Зберігається	AEAD виявляє будь-яку зміну записів TLS у межах сесії.
Автентичність сервера	Порушується	Клієнт не знає, з ким встановив ключ.

Властивість	Стан	Пояснення
Конфіденційність і цілісність щодо активного атакувальника	Порушуються	Оскільки ключ може бути узгоджено з атакувальником, шифрування захищає дані «до нього», а не від нього.
Forward Secrecy	Формально є, але втрачає сенс	Ефемерні ключі захищають від запису трафіку, але не від MITM у реальному часі.

Таблиця 6 — Властивості захищеного каналу після вимкнення перевірки сертифіката

6.3. Чи може зловмисник прочитати або модифікувати дані

Пасивний зловмисник — ні. Активний зловмисник, здатний перехопити з'єднання, — **так**: він надає клієнту власний (самопідписаний або будь-який) сертифікат, який клієнт приймає без перевірки, встановлює окрему TLS-сесію з реальним сервером і ретранслює дані, маючи змогу читати й змінювати їх у відкритому вигляді, зокрема паролі, токени API, відповіді сервера (з можливістю впровадження шкідливих даних, оновлень тощо). Ні клієнт, ні сервер цього не помітять. Саме так працюють інструменти mitmproxy, Burp Suite, а також шлюз із п. 3.2 — з тією різницею, що там клієнт свідомо довіряє кореню шлюзу.

6.4. Умови, за яких можлива атака MITM

- Атакувальник контролює мережевий шлях або може перенаправити трафік: публічна чи підроблена Wi-Fi-точка (evil twin), ARP-spoofing у локальній мережі, DNS-spoofing / отруєння кешу, компрометований маршрутизатор, провайдер, BGP-перехоплення, шкідливий проксі.
- Клієнт не перевіряє сертифікат (як у ситуації), або перевіряє неповністю (лише ланцюжок без імені хоста), або довіряє кореневому ЦС, яким керує атакувальник (встановленому шкідливим ПЗ чи адміністратором).
- Відсутні додаткові механізми: HSTS (для браузерів), certificate pinning, взаємна автентифікація (mTLS).

6.5. Як правильно усунути початкову проблему

1. Діагностувати причину: `openssl s_client -connect host:443 -servername host -showcerts` і проаналізувати код помилки (20 — не знайдено видавця, 10 — прострочено, 19/18 — самопідписаний, невідповідність імені).
2. Усунути її на стороні сервера: встановити сертифікат від довіреного ЦС (напр., Let's Encrypt), налаштувати передачу повного ланцюжка з проміжними сертифікатами, оновити прострочений сертифікат, включити в SAN усі потрібні імена.
3. Якщо використовується внутрішній/корпоративний ЦС — не вимикати перевірку, а додати **саме цей кореневий сертифікат** до довірених для застосунку (напр., `verify="/path/corp-ca.pem"` у `requests`, `SSL_CERT_FILE`, власний `SSLContext`), обмежуючи довіру лише ним.

4. Для тестового середовища зі самопідписаним сертифікатом — створити власний тестовий ЦС і довіряти йому лише в тестовій конфігурації; ніколи не залишати `verify=False` у продакшн-кодi.
5. Оновити сховище кореневих сертифікатів клієнта (пакели `ca-certificates`, `certifi`).
6. Для мобільних/критичних застосунків — розглянути `certificate/public key pinning`, використовувати TLS 1.2+ (переважно 1.3), додати в CI статичний аналіз на наявність вимкненої перевірки (напр., правило Bandit B501).

7. Висновки

1. Практичне дослідження показало, що з'єднання встановлюються за TLS 1.3 з AEAD-шифрами та ефемерним обміном ключами X25519, що забезпечує Forward Secrecy. За даними незалежного сканування `github.com` підтримує також TLS 1.2 лише з ECDHE-наборами і відхиляє TLS 1.1 та старіші версії; сертифікат має короткий строк дії (≈ 90 діб), містить SCT для Certificate Transparency, а сайт застосовує HSTS із preload.

2. Під час роботи виявлено, що в середовищі виконання трафік проходить через шлюз TLS-інспекції: клієнт отримував сертифікати, видані внутрішнім ЦС, що відрізнялися від справжніх (ключ, видавець, строк дії) і не проходили перевірку щодо публічного сховища Mozilla. Це практичне підтвердження того, що зашифрованість каналу нічого не каже про те, з ким цей канал встановлено; автентичність сервера визначається виключно перевіркою сертифіката та складом довірених кореневих ЦС.

3. TLS 1.3 усунув цілі класи атак (POODLE, BEAST, Lucky 13, FREAK, Logjam) за рахунок вилучення CBC, статичного RSA, експортних шифрів і вбудованого захисту від downgrade. Досліджена атака POODLE для TLS 1.3 неактуальна, однак демонструє небезпеку збереження застарілих протоколів «для сумісності».

4. Механізми відкликання (CRL, OCSP) мають суттєві обмеження, тому галузь переходить до коротких строків дії сертифікатів і прозорості видачі (CT), а HSTS доповнює TLS захистом від SSL stripping.

5. Вимкнення перевірки сертифіката зберігає лише захист від пасивного прослуховування, але повністю руйнує автентичність і відкриває шлях до MITM. Правильне рішення — усунути причину помилки (ланцюжок, строк дії, SAN) або явно довіряти лише потрібному ЦС, не вимикаючи перевірку.

8. Перелік використаних джерел

1. Rescorla E. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. IETF, 2018. URL: <https://www.rfc-editor.org/rfc/rfc8446>
2. Dierks T., Rescorla E. The TLS Protocol Version 1.2. RFC 5246. IETF, 2008. URL: <https://www.rfc-editor.org/rfc/rfc5246>
3. Moriarty K., Farrell S. Deprecating TLS 1.0 and TLS 1.1. RFC 8996. IETF, 2021. URL: <https://www.rfc-editor.org/rfc/rfc8996>
4. Barnes R. et al. Deprecating Secure Sockets Layer Version 3.0. RFC 7568. IETF, 2015.
5. Cooper D. et al. Internet X.509 PKI Certificate and CRL Profile. RFC 5280. IETF, 2008.
6. Santesson S. et al. X.509 Internet PKI Online Certificate Status Protocol — OCSP. RFC 6960. IETF, 2013.
7. Laurie B., Langley A., Kasper E. Certificate Transparency. RFC 6962 (2013); Certificate Transparency Version 2.0. RFC 9162 (2021).
8. Hodges J., Jackson C., Barth A. HTTP Strict Transport Security (HSTS). RFC 6797. IETF, 2012.
9. Moeller B., Langley A. TLS Fallback Signaling Cipher Suite Value (SCSV). RFC 7507. IETF, 2015.
10. Möller B., Duong T., Kotowicz K. This POODLE Bites: Exploiting the SSL 3.0 Fallback. Google Security Advisory, 2014.
11. NIST SP 800-52 Rev. 2. Guidelines for the Selection, Configuration, and Use of TLS Implementations. 2019.
12. OpenSSL 3.0 Documentation: openssl-s_client(1), openssl-x509(1). URL: <https://docs.openssl.org/3.0/>
13. Qualys SSL Labs API Documentation v3. URL: <https://github.com/ssllabs/ssllabs-scan/blob/master/ssllabs-api-docs-v3.md>
14. CA/Browser Forum. Baseline Requirements for the Issuance and Management of Publicly-Trusted TLS Server Certificates. URL: <https://cabforum.org/baseline-requirements/>
15. MDN Web Docs. Strict-Transport-Security. URL: <https://developer.mozilla.org/docs/Web/HTTP/Headers/Strict-Transport-Security>